

Contents

Language and Automaton	2
Boolean Function	2
Language	2
DFA	2
NFA	5
Regular Expression	6
PDA	9
CFL	9
Grammar	10
CFG	10
Turing Machine	10
NAND-TM	10
NAND-RAM	12
Universal Turing Machine	12
Uncomputable Problem	12
Halting problem	13
Rice's Theorem	14
Quine Problem	15
Recursion Theorem	16
Fixed Point Theorem	18
Godel's Incompleteness Theorem	18
Quantified Integer Statement	19
Running Time	20
Running Time of Turing Machine	20
P and EXP	20
Running Time of NAND-RAM Program	20
TM and NAND-RAM	21
Universal TM and Universal NAND-RAM Program	21
Time Hierarchy Theorem	21
Complexity of Circuit and Time	22
polynomial-time Reductions	24
NP	26
Random Algorithm and BPP	29
多项式等式验证	29
矩阵乘法验证	30
随机计算模型	31
BPP	31
BPP 与其他复杂类的关系	32
伪随机生成器	33

Language and Automaton

Boolean Function

我们将输入和输出都扩展至任意长度, 即 $\{0, 1\}^* \rightarrow \{0, 1\}^*$. 而如果要讨论这类函数的可计算性, 只需要考虑输出长度为 1 的函数即可, 这样的函数叫做布尔函数(Boolean function). 意即:

Booleanizing general functions

对于任意的函数 $F: \{0, 1\}^* \rightarrow \{0, 1\}^*$, 存在一个布尔函数 $BF: \{0, 1\}^* \rightarrow \{0, 1\}$, 使得一个用于计算函数 BF 的 Python 程序可以被用来计算函数 F , 反之亦然。

 **proof** 考虑函数 $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$, 设计一个布尔函数 $bf: \{0, 1\}^* \rightarrow \{0, 1\}$

$$bf(x, i, c) = \begin{cases} f(x)_i & \text{if } c = 0, i < |f(x)| \\ 1 & \text{if } c = 1, i < |f(x)| \\ 0 & \text{if } i \geq |f(x)| \end{cases}$$

那我们可以利用函数 bf 求解函数 f , 操作如下:

Algorithm 1: Compute f by bf

```
1: procedure F( $x$ )
2:    $\triangleright$  Input:  $x \in \{0, 1\}^*$ 
3:    $\triangleright$  Output:  $f(x) \in \{0, 1\}^*$ 
4:    $res \leftarrow []$ 
5:    $i \leftarrow 0$ 
6:   while  $BF(x, i, 1)$  do
7:      $\triangleright res.append(BF(x, i, 0))$ 
8:      $i \leftarrow i + 1$ 
9:   end
10:  return  $res$ 
11: end
```

Language

可以看到的是: 一个布尔函数自然会对应一个字符串的集合, 这个集合我们称之为 language, 其是那些函数值为 1 的输入的集合。

Boolean function	Language
$f: \{0, 1\}^* \rightarrow \{0, 1\}$	$\implies A_f = \{x \in \{0, 1\}^* f(x) = 1\}$
$f_{A(x)} = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{if } x \notin A \end{cases}$	$\longleftarrow A$

DFA

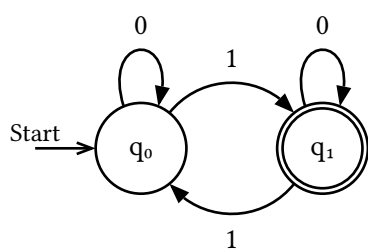
At a high level, an algorithm is a recipe for computing an output from an input via a combination of the following steps:

- Read a bit from the input
- Update the state (working memory)
- Stop and produce an output

比如一个简单的用于计算 XOR 函数的 Python 程序:

```
def XOR(X):
    ''' Takes list X of 0's and 1's
        Output 1 if the number of 1's in X is odd and outputs 0 otherwise
    '''
    result = 0
    for i in range(len(X)):
        result = (result + X[i]) % 2
    return result
```

在这个程序中，其每一步都读取 X 的一个位并更新状态，直到输出一个结果。这样的程序我们称之为 single-pass constant-memory algorithm, 这样的算法也被称作 DFA: Deterministic Finite Automaton, 确定性有限自动机，又叫做有限状态机。DFA 可以用一个状态转换图表示，上面的 XOR 程序为例：



💡 DFA

一个 DFA 可以看作一个四元组：DFA $M = K, s, F, \delta$, 其中：

- K : a finite set of states
- s : initial state
- F : a finite set of accepting states
- δ : transition function, as $K \times \{0, 1\} \rightarrow K$

对于一个 n 位的输入 $x_0x_1 \dots x_{n-1}$, 其计算过程为：

- $s_0 = s$
- $s_i = \delta(s_{i-1}, x_{i-1})$
- $s_n = \delta(s_{n-1}, x_{n-1})$
- if $s_n \in F$: accept
- else: reject

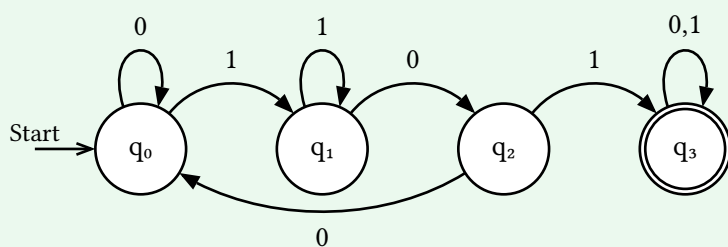
- DFA M compute 一个布尔函数 f , 如果 M accept $x \iff f(x) = 1$
- M decides 一个语言 A , 如果 M accept $x \iff x \in A$ 。

集合 $L(M)$ 包含了那些被 M 接受的字符串，这样的字符串我们称之为正则语言 (Regular Language)。

📄 example

设计一个 DFA，其能判定语言 $L: \{w \in \{0, 1\}^* \mid w \text{ contains } 101 \text{ as a substring}\}$

✅ 答案



并非所有的语言都是正则的，因为 DFA 可以被编码，说明 DFA 的数目是可数的，而 language 是不可数的，因此存在 language 是 non-regular 的。正则语言对下面的运算是封闭的：

- 并
- 交
- Kleene 闭包
- 补
- 连接

proof

由于 DFA 与 NFA 等价，为了证明方便，我们在证明并、交、补时使用 DFA，而在证明闭包和连接时构造 NFA。

- Union ($L_1 \cup L_2$): Let DFA A decides L_1 , DFA B decides L_2 , A, B 分别定义如下:

- $A = (K_A, s_A, F_A, \delta_A)$
- $B = (K_B, s_B, F_B, \delta_B)$

构造 DFA $M = (K, s, F, \delta)$ (Product Construction):

- $K = K_A \times K_B$ (状态集为笛卡尔积)
- $s = (s_A, s_B)$
- $\delta((q_A, q_B), a) = (\delta_A(q_A, a), \delta_B(q_B, a))$
- $F = \{(q_A, q_B) \mid q_A \in F_A \vee q_B \in F_B\}$

该 DFA 并行模拟 A 和 B ，只要 A 或 B 中有一个接受， M 即接受。

- Intersection ($L_1 \cap L_2$): 与并运算的构造完全相同，唯一的区别在于接受状态集合 F :

- $F = \{(q_A, q_B) \mid q_A \in F_A \wedge q_B \in F_B\}$

只有当 A 和 B 同时处于接受状态时， M 才接受。

- Complement ($\Sigma^* \setminus L_1$): Let DFA $A = (K_A, s_A, F_A, \delta_A)$ decides L_1 . 构造 DFA $M = (K_M, s_M, F_M, \delta_M)$:

- $K_M = K_A$
- $s_M = s_A$
- $\delta_M = \delta_A$
- $F_M = K_A \setminus F_A$

M 接受所有 A 拒绝的字符串，拒绝所有 A 接受的字符串。即 $L(M) = \overline{L(A)}$ 。

- Concatenation ($L_1 \cdot L_2$): 为了证明连接运算的封闭性, 我们利用 NFA 的 ϵ 转移特性。Let NFA N_1 decides L_1 , NFA N_2 decides L_2 (假设状态集互不重叠)。构造 NFA $M = (K, s, F, \delta)$:
 - $K = K_{\{N_1\}} \cup K_{\{N_2\}}$
 - $s = s_{\{N_1\}}$
 - $F = F_{\{N_2\}}$
 - δ : 包含 N_1 和 N_2 原有的所有转移, 并增加如下 ϵ 转移:

$$\forall q \in F_{N_1}, \text{ add transition } q \xrightarrow{\epsilon} s_{N_2}$$

这就相当于: 当 N_1 读完前半段字符串到达接受状态时, 我们可以非确定性地“跳转”到 N_2 的开始状态去处理剩余的字符串。

- Kleene Star (L_1^*): 同样利用 NFA 构造。Let NFA $N_1 = (K_1, s_1, F_1, \delta_1)$ decides L_1 . 构造 NFA $M = (K, s, F, \delta)$:
 - 引入一个新的起始状态 s_{new} 。
 - $K = K_1 \cup \{s_{new}\}$
 - $s = s_{new}$
 - $F = F_1 \cup \{s_{new}\}$ (因为 $\epsilon \in L_1^*$, 所以新起点也是接受态)
 - δ : 包含原有的转移, 并增加:
 1. $s_{new} \xrightarrow{\epsilon} s_1$ (允许开始进入原机器)
 2. $\forall q \in F_1, q \xrightarrow{\epsilon} s_1$ (允许在原机器接受后, 跳回开头重复匹配)

NFA

NFA 是 DFA 的扩展, 其主要区别在于引入了“非确定性”:

- 多重路径: 对于同一个输入符号, NFA 可以转移到多个状态。
- ϵ 转移: NFA 可以在不读入字符的情况下进行状态转移

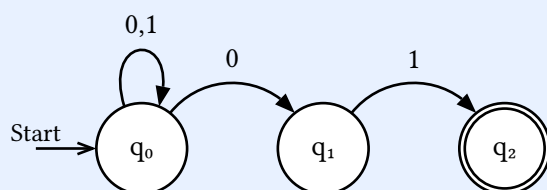
💡 NFA

一个 DFA 可以看作一个四元组: DFA $M = K, s, F, \delta$, 其中:

- K : a finite set of states
- s : initial state
- F : a finite set of accepting states
- δ : transition function, as $K \times \{0, 1, \epsilon\} \rightarrow K$

可以看到只有转移函数部分是不同的。

 **example** 识别以“01”结尾的字符串的 NFA:



NFA 似乎看起来比 DFA 更强, 因为它可以“猜测”正确的路径。但是, 可以证明它们能判定的语言是相等的。

L(DFA) = L(NFA)

任何 NFA 识别的语言都可以被一个 DFA 识别。

 **proof** 子集构造法 (Subset Construction): 给定 NFA $N = (K_N, \Sigma, \Delta_N, s_N, F_N)$, 我们构造一个 DFA $M = (K_M, \Sigma, \Delta_M, s_M, F_M)$:

- 状态集: $K_M = P(K_N)$ 。DFA 的每个状态是 NFA 状态的一个子集。
- 初始状态: $s_M = E(\{s_N\})$, 其中 $E(R)$ 表示从集合 R 出发经 ε 转移能到达的所有状态集合。
- 转移函数: 对于 $S \in K_M$ 和 $a \in \Sigma$,

$$\Delta_{M(S,a)} = \bigcup_{q \in S} E(\Delta_{N(q,a)})$$

- 接受状态: $F_M = \{S \in K_M \mid S \cap F_N \neq \emptyset\}$ (只要子集中包含至少一个 NFA 的接受状态)。

通过归纳法可证 $L(M) = L(N)$ 。

Regular Expression

一个语言是正则的当且仅当其可被一个正则表达式(Regular Expression, RE)表示。

Regular Expressions

正则表达式 R 是通过以下递归规则定义的:

1. a : for $a \in \Sigma$
2. ε : 空串
3. $\emptyset$: 空集
4. $(R_1 \cup R_2)$: 并运算 (Union)
5. $(R_1 \cdot R_2)$: 连接运算 (Concatenation)
6. (R_1^*) : Kleene Star (0 次或多次重复)

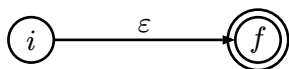
这样看来, 正则表达式和 NFA 应当可以相互转化, 事实上也是的。

- RE $\implies$ NFA (Thompson 构造法)

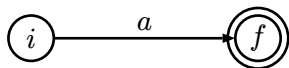
利用 CeTZ 绘制精确的结构图。我们将子表达式 s 和 t 对应的 NFA 抽象为从状态 i' 到 f' 的一次转移 $N(s)$ 。

- 两条基本规则:

- 表达式 ε :

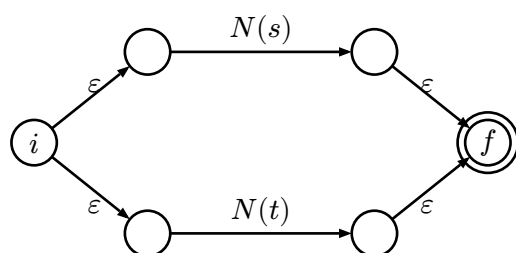


- 表达式 a :

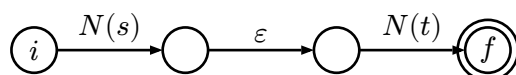


- 归纳部分:

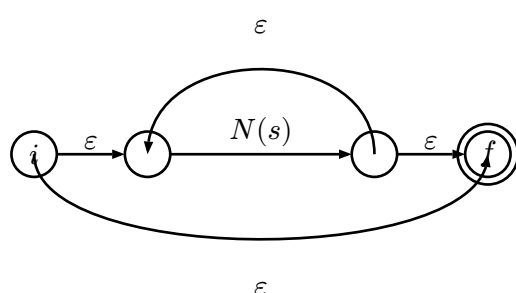
- 表达式 $s \mid t$ (Union), 并行对称结构:



- 表达式 st (Concatenation), 串行结构:



- 表达式 s^* (Kleene Star), 回环结构:



• NFA $\implies$ RE(状态消去法)

对于一个 NFA, 可以按照以下步骤将其转换成 RE:

1. 标准化 NFA:

- 唯一初始状态: 确保只有一个初始状态, 如果有多个, 则添加一个新的初始状态 q_{start} , 并用 ϵ 转移到原初始状态
- 唯一接受状态: 确保只有一个接受状态, 如果有多个, 则添加一个新的接受状态 q_{final} , 并用 ϵ 从原接受状态转移到 q_{final}
- 无进入初始状态的边: 初始状态不能有进入边, 除非是自环
- 无离开接受状态的边: 接受状态不能有离开边, 除非是自环

2. 选择状态消去顺序

可以按照任意顺序消去中间状态, 通常不影响最终结果的正确性, 但可能影响表达式的复杂度

3. 消去状态 q

对于要消去的状态 q , 考虑:

- 进入边: 设为从状态 p_i 到 q , 标记为 R_{in}^i
- 离开边: 设为从 q 到状态 p_j , 标记为 R_{out}^j
- 自环: q 上的自环, 标记为 R_{loop}

对于每一对 (p_i, p_j) , 计算 p_i 到 p_j 的新路径表达式:

$$R_{new} = R_{in}^i \cdot (R_{loop})^* \cdot R_{out}^j$$

如果从 p_i 到 p_j 已存在一个路径表达式, 记为 R_{old} , 则将其合并, 合并后的表达式为:

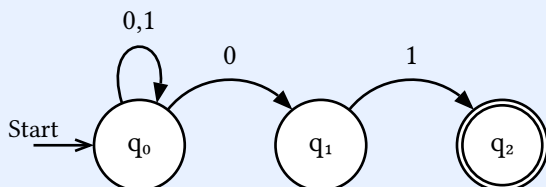
$$R_{old} \mid R_{new}$$

4.重复步骤 3, 直到最后只剩下 q_{start} 和 q_{final}

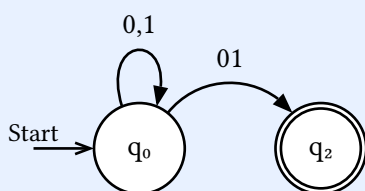
5.最终图中, 从 q_{start} 到 q_{final} 的边, 即为最终的 RE, 如果存在多条边, 则用选择运算合并, 如果存在自环, 则用 Kleene 星处理

example

考虑下面这个简单的 NFA:



消去中间状态 q_1 , 得到如下的 NFA:



最终结果即为:

$$(0|1)^*01$$

证明一个语言是正则的, 我们可以通过构造出其对应的 DFA/NFA/RE 来证明, 而证明一个语言不是正则的, 我们则可以借助泵定理 (Pumping Theorem):

Pumping Theorem

若 L 是正则语言, 则存在一个整数 $p \geq 1$ (Pumping Length), 使得任意 $w \in L$ 且 $|w| \geq p$, 都可以被分割成三部分 $w = xyz$, 满足:

1. $|y| > 0$ (中间部分非空)
2. $|xy| \leq p$ (前两部分长度受限)
3. $\forall i \geq 0, xy^iz \in L$ (中间部分可以被“泵”任意次)

proof

鸽巢原理 (Pigeonhole Principle): 设 DFA M 有 p 个状态。对于任意长度 $n \geq p$ 的字符串, 其经过的状态序列长度为 $n + 1$ 。根据鸽巢原理, 在处理前 p 个字符的过程中, 必然至少有一个状态 q 被重复访问了两次。我们将这两次访问之间的部分记为 y 。因为 M 是确定性的, 循环 y 任意次 (y^i) 都会回到状态 q , 最后仍能到达接受状态。

example

试利用 Pumping Theorem 证明 $\{0^n 1^n : n \geq 0\}$ 不是正则的

proof

使用反证法 (Proof by Contradiction):

1. 假设 L 是正则的, 设 p 为其 Pumping Length。
2. 选取字符串 $w = 0^p 1^p$ 。显然 $w \in L$ 且 $|w| = 2p \geq p$ 。
3. 根据 Pumping Lemma, w 可以写作 xyz , 满足 $|xy| \leq p$ 且 $|y| > 0$ 。
4. 因为 $|xy| \leq p$, 且 w 以前 p 个 0 开头, 所以 y 必然只包含 0 (即 $y = 0^k, k \geq 1$)。
5. 取 $i = 2$ (pump up), 考虑 $w' = xy^2z$ 。
 - w' 中 0 的数量为 $p + k$ 。
 - w' 中 1 的数量仍为 p 。
 - 因为 $k \geq 1$, 所以 0 的数量不等于 1 的数量。
6. $w' \notin L$, 但这与 Pumping Lemma 的结论矛盾。
7. 因此, 假设不成立, L 不是正则语言。

PDA

DFA 加上一个无限容量的栈 (Stack) 就构成了下推自动机 (PDA)。

 **PDA Definition** PDA 是一个六元组 $(K, \Sigma, \Gamma, \Delta, s, F)$:

- Γ : 栈字母表 (Stack Alphabet)
- 转移函数 $\Delta: K \times (\Sigma \cup \{\epsilon\}) \times (\Gamma \cup \{\epsilon\}) \rightarrow P(K \times (\Gamma \cup \{\epsilon\}))$ 如 (q, a, x, p, y)

即: 在状态 q , 读取输入 a , 弹出栈顶 $x \rightarrow$ 转移到状态 p , 压入栈顶 y 。

Algorithm 2: PDA Execution Logic

```
1: procedure RUNPDA( $w$ )
2:    $\triangleright$  Stack allows Last-In-First-Out memory
3:   "stack"  $\leftarrow$  Empty
4:   "current_state"  $\leftarrow$   $s$ 
5:   while input not empty or epsilon-transition possible do
6:      $\triangleright$  read_input, pop_stack
7:     if transition defined then
8:        $\triangleright$  Update current_state
9:        $\triangleright$  Push new_symbol
10:    else
11:      return Reject
12:    end
13:  end
14:  if current_state in  $F$  then
15:    return Accept
16:  else
17:    return Reject
18:  end
19: end
```

CFL

上下文无关语言。一个语言是上下文无关的 (CFL), 当且仅当存在一个 PDA 识别该语言。

$$L(G) = L(PDA)$$

需要注意的是：CFL 对交集不封闭

Grammar

CFG

Turing Machine

💡 Turing Machine

A Turing machine $M = (K, \Sigma, s, \delta)$, where:

- K : a finite set of states
- Σ : a finite set of symbols, which must include $\{0, 1, \triangleright, \sqcup\}$, where $\triangleright$ means the leftest of the tape (tape start with $\triangleright$), $\sqcup$ means empty part of the tape
- s : initial state
- δ : transition function $: K \times \Sigma \longrightarrow K \times \Sigma \times \{L, R, S, H\}$, which means: for a state $k \in K$, and a symbol $a \in \Sigma$, the transition function returns a triple (k', a', d) , k' is the next state, a' is the output symbol, and d is the direction of the head. Some triple (k, a) doesn't have a corresponding triple in δ , which means that the machine will halt or refuse
 - L : move the head to the left
 - R : move the head to the right
 - S : stay at the same place
 - H : halt

当图灵机能够停机时,图灵机会有输出,记为 $M(x) = T[1] \cdots T[i]$, 其中 $T[i+1]$ 为第一个空白格 $\sqcup$, 若图灵机不停机,则记作 $M(x) = \perp$

💡 compute function

A Turing Machine M computes a function $f : \{0, 1\}^* \longrightarrow \{0, 1\}^*$, if $\forall x \in \{0, 1\}^*, M(x) = f(x)$

即:可计算函数等价于一台可以停机的图灵机.

NAND-TM

NAND-TM 可以看作是 NAND-CIRC 的扩展,其在后者的基础上添加了无限数组和循环(这里的循环可以和输入相关),使其能够处理任意长度的输入.

可以简单回顾一下 NAND-CIRC 语言:每一行都形如 $[a = \text{NAND}(b, c)]$

NAND-TM 语言:

- 包括两种数据类型:
 - i : integer value, 用作数组的下标
 - other: boolean value
- 包括两类变量:
 - scalars: boolean value
 - arrays: 所有的数组都无限长度,但共用一个 index
- 输入 X 和输出 Y 也都是数组

- 程序的最后一行有一个特殊指令:MODANDJUMP:modify i and jump to the first line.其有两个输入:a 和 b.
 - $a = 1$ and $b = 1: i = i + 1$
 - $a = 1$ and $b = 0: i = i - 1$
 - $a = 0$ and $b = 1: i = i$
 - $a = 0$ and $b = 0: \text{halt}$
- default value:除了输入外,所有变量的初始值都是 0.为了区分数组的某一位究竟是未被写入过(初始 0)还是被显式写入了 0,我们可以定义一个指示数组 Valid,Valid[i] = 1 时,Array[i]的值才有效

NAND-TM example

我们可以构造一个 NAND-TM 程序,计算函数 $Y = X + 1$:

而可以证明的是:一个 NAND-TM 程序等价于一台图灵机:

NAND-TM $\Leftrightarrow$ Turing Machine

NAND-TM program 和 Turing Machine 中的元素有以下的对应关系:

NAND-TM	Turing Machine
scalar variable	states
array	tape
i	head
i:-1/+1/0	head:left/right/stay

proof

- Turing Machine $\Rightarrow$ NAND-TM:
 - 我们可以用 $\lceil \log k \rceil$ 个 scalar variable 来编码图灵机的 k 个状态
 - 用 $\lceil \log |\Sigma| \rceil$ 个数组编码图灵机的 tape Σ
 - 用两个 scalar variable 来编码图灵机的读写头的操作:

$$\{L, R, S, H\} \rightarrow \{01, 11, 10, 00\}$$
 - 经过上面的编码,图灵机的转移函数 $\delta: K \times \Sigma \rightarrow K \times \Sigma \times \{L, R, S, H\}$ 就被编码成了输入输出均为 0-1 字符串的函数 $\delta': \{0, 1\}^{\lceil \log K \rceil + \lceil \log |\Sigma| \rceil} \rightarrow \{0, 1\}^{\lceil \log K \rceil + \lceil \log |\Sigma| \rceil + 2}$. 函数 δ' 的输入和输出都是固定的,即其为一个 finite function,故存在一个 NAND-CIRC(记作 P-CIRC)可以用于计算 δ' ,通过指令 MODANDJUMP 读取输出的最后两位,即可以得到图灵机读写头的位置变化
- NAND-TM $\Rightarrow$ Turing Machine: 与上面的步骤类似,用图灵机中的元素来模拟 NAND-TM 中的元素.

可以添加一些语法糖,让 NAND-TM 程序更接近现代程序语言:

- GOTO:跳转到某个行
- WHILE:循环
- 前文提到的 NAND-TM 只有一个 index,实则可以改造成能够接受多个 index
- 前文提到的 NAND-TM 的数组都是一维数组,实则可以改造成多维数组

NAND-RAM

NAND-RAM(Random Access Machine)是对 NAND-TM 的进一步扩展,旨在模拟现代计算机的随机访问内存特性,使其编程更符合直觉

- 整数变量:变量可以存储有界整数,而非仅限于布尔值,整数范围受限于计算机的字长
- 随机访问数组:数组访问指令不再局限于固定索引*i*,可以通过形如 $A[\text{expr}]$ 的语法,用任意整数表达式 expr 的结果作为索引来读取或写入数组,这是与 NAND-TM 的关键区别
- 扩展指令集:在 NAND 的基础上,增加了整数算数运算,(如加,减,乘,除,比较),位操作和条件跳转等高级指令

同样的,我们也可以证明:NAND-RAM 程序 $\Leftrightarrow$ NAND-TM

事实上,图灵机和许多模型都是等价的,其中甚至包括 Minecraft 的红石模型

The Church-Turing Thesis

判断:任何在算法上(即通过有效方法)可计算的函数,都能被一台图灵机计算

- 这是一个哲学观点,而非一个被证明了的数学定理.其基于对“算法”;“有效过程”的直观定义以及所有已知计算模型(λ 演算,递归函数,现代编程语言等)都被证明与图灵机等价这一事实
- 其确立了图灵机作为计算通用模型的地位,后续提到的“可计算”,若无特殊说明,均指“图灵机可计算”

Universal Turing Machine

图灵机既可以看作软件也可以看作硬件,定义一台图灵机后,既可以用逻辑语言来描述它,又可以用晶体管实现它.然而我们并不希望对于任意问题都定义一个特定的图灵机来解决它,因此通用图灵机(Universal Turing Machine)应运而生.

Universal Turing Machine

通用图灵机是一台特殊的图灵机 U ,其输入是另一台图灵机 M 的描述(编码)以及 M 的输入 x .

$$U(\langle M \rangle, x) = \begin{cases} M(x) & \text{if } M \text{ halt on } x \\ \perp & \text{if } M \text{ does not halt on } x \end{cases}$$

- U 通过模拟 M 在 x 上的运行来工作
- 这证明了图灵机可以像软件一样被存储和解释,奠定了可编程计算机的理论基础
- U 本身是一台固定的图灵机,却能模拟任何其他图灵机,展现了计算的通用性

Uncomputable Problem

然而,图灵机尽管如此强大,但仍有许多问题其无法解决.



$\exists$ a function $F^* : \{0, 1\}^* \rightarrow \{0, 1\}$ is not computable by any Turing Machine, and this type of function is called “uncomputable”

Proof 1

the set of boolean function is uncountable, and the set of Turing machines is countable (because Turing Machine can be encoded as a string).

The number of boolean function $\gg$ the number of Turing Machines, so there must be a function F^* that is not computable by any Turing Machine.

Proof 2

let $F^*(x)$ be:

$$F^*(x) = \begin{cases} 0 & \text{if } x \text{ is a encoding of a Turing Machine } M \text{ and } M(x) = 1 \\ 1 & \text{otherwise (} x \text{ is not valid encoding or } M(x) \neq 1 \text{ or } M(x) = \perp \text{)} \end{cases}$$

Assertion: F^* is uncomputable

采用反证法:假设存在图灵机 M^* 可以计算 F^* , 即对任意 x , $M^*(x) = F^*(x)$ 且总停机, 考虑将 M^* 自身的编码 $\langle M^* \rangle$ 作为输入:

- 假设 $M^*(\langle M^* \rangle) = 1$, 则 $F^*(\langle M^* \rangle) = 0$, 产生矛盾
- $M^*(\langle M^* \rangle) = 0$ or $\perp$, 由于 M^* 总停机, 所以只能是 $M^*(\langle M^* \rangle) = 0$, 则此时 $F^*(\langle M^* \rangle) = 1$, 产生矛盾

故不存在这样的图灵机 M^* 能够计算 F^*

该证明方法是经典的对角化论证

Halting problem

停机问题(Halting problem)是一个著名的无法被图灵机解决的问题: Given a TM M and a string x , does M halt on x ?

$$\text{HALT}(M, x) = \begin{cases} 1 & \text{if } M \text{ halts on } x \\ 0 & \text{otherwise} \end{cases}$$

HALT is uncomputable

假设存在图灵机 M_{HALT} 可以计算 HALT 函数, 则我们可以构造一台图灵机 M^* 来计算之前定义的, 被证明过不可计算的函数 F^* :

1. M^* 的输入是字符串 x
2. M^* 首先检查 x 是否是某个图灵机 M_x 的有效编码, 若不是, 则直接输出 1
3. 若 x 是有效编码, 则 M^* 调用假设存在的 $M_{\text{HALT}}(M_x, x)$
 - 若 $M_{\text{HALT}}(M_x, x) = 0$ (M_x 在 x 上不停机), 则 M^* 输出 1
 - 若 $M_{\text{HALT}}(M_x, x) = 1$ (M_x 在 x 上停机), 则 M^* 可以模拟运行 $M_x(x)$, 得到结果后检查其是否为 1, 并根据结果决定 M^* 最后的输出:
 - 若 $M_{x(x)} = 1$, 则 M^* 输出 0
 - 若 $M_{x(x)} \neq 1$, 则 M^* 输出 1

这样的 M^* 总是停机, 且能正确计算 F^* , 而我们已经证明过 F^* 是不可计算的, 故产生矛盾

因此假设错误,停机问题是不可计算的

上面的证明方法其实运用了规约的思想.

💡 Reduction

A reduction from F to G is a computable function $R : \{0, 1\}^* \rightarrow \{0, 1\}^*$, such that $F(x) = G(R(x))$

即:如果我们能通过解决问题 B 来解决问题 A ,则称问题 A 可以归约到问题 B

📄 Reduction Lemma

Suppose that there is a reduction R from F to G .

- If G is computable, then F is computable.
- If G is uncomputable, then F is uncomputable.

从停机问题出发,我们可以得到更多无法被图灵机解决的问题,可以用规约的思想证明其不可计算性:,如

- HALTONZERO: Given a TM M and a string x , does M halt on $0s$?
- ZEROFUNCTION: Given a TM M and a string x , is $M(x) = 0 \forall x \in \{0, 1\}^*$?

我们可以证明这两个问题都是不可计算的:

🔍 HALTONZERO is uncomputable

$\text{HALTONZERO}(M) = 1$ iff M halts on $0s$

给定一个 HALT 问题的实例:图灵机 M 和输入字符串 x ,我们需要判断 M 是否在 x 上停机

构造图灵机 M_x :

- 忽略 M_x 自身的输入
- 模拟图灵机 M 在 x 上的行为

若 M 在 x 上停机,那么 M_x 在任何输入(包括 $0s$)上都会停机,则 $\text{HALTONZERO}(M_x) = 1$; 若 M 在 x 上不停机,则 M_x 在任何输入上都不停机,则 $\text{HALTONZERO}(M_x) = 0$

所以: $\text{HALT}(M, x) = \text{HALTONZERO}(M_x)$, 因为停机问题不可计算,所以 HALTONZERO 问题也不可计算

🔍 ZEROFUNCTION is uncomputable

$\forall x \in \{0, 1\}^*, \text{ZEROFUNCTION}(M) = 0$

Rice's Theorem

功能等价性: 如果对于任意的输入 x ,都有 $M(x) = M'(x)$, 则称图灵机 M 和 M' 功能等价 (functionally equivalent)

A property (function) $P : \{0, 1\}^* \rightarrow \{0, 1\}$ is semantic if $P(M) = P(M')$ for any pair of functionally equivalent TM M and M' .

Rice's theorem

设 $P : \{0, 1\}^* \rightarrow \{0, 1\}$ 是一个关于图灵机描述的语义性质,且满足:

- 非平凡性(nontriviality):存在图灵机 M_{yes} ,使得 $P(M_{\text{yes}}) = 1$,也存在图灵机 M_{no} ,使得 $P(M_{\text{no}}) = 0$
 - ▶ 满足非平凡性的性质:如 P :图灵机在空输入上停机.有的图灵机在空输入上停机,也有不停机
 - ▶ 不满足非平凡性的性质:如 P :图灵机有至少 1 个状态.所有图灵机都满足这条性质,所以这条性质是平凡的
- 语义性(semantic):若两台图灵机 M 和 M' 计算相同的函数(即功能等价),则 $P(M) = P(M')$
 - ▶ 满足语义性的性质:如 P :图灵机在空输入上停机.如果两个图灵机计算相同的函数,则它们在空输入上的行为也相同.
 - ▶ 不满足语义性的性质:如 P :图灵机有超过 50 个状态.即使两个图灵机计算相同的函数,由于实现不同,它们可能有不同的状态数.

那么,性质 P 是不可判定的,即函数 P 是不可计算的

Proof

我们可以通过将停机问题规约到任意非平凡的语义性质 P 来证明这一定理.

假设 P 是非平凡的语义性质,不失一般性,假设对于永远不停机的图灵机 M_{∞} ,有 $P(M_{\infty}) = 0$,同时,取一图灵机 M_{yes} ,满足 $P(M_{\text{yes}}) = 1$

构造规约 R :

- 输入:一个停机问题的实例 (M, x)
- 输出:一台新图灵机 M_x' 的描述

构造 M_x' :

- M_x' 的输入是字符串 y
- 对于输入 y :
 - ▶ 在一条额外的纸带上模拟运行 $M(x)$
 - ▶ 若 $M(x)$ 停机,则丢弃这个模拟结果,转而模拟运行 $M_{\text{yes}}(y)$,并输出其输出
 - ▶ 若 $M(x)$ 不停机,则 M_x' 也将不停机

在这个构造下:

- 若 $M(x)$ 停机,则对于任意输入 y , $M_x'(y) = M_{\text{yes}}(y)$,因此 $P(M_x') = P(M_{\text{yes}}) = 1$
- 若 $M(x)$ 不停机,则对于任意输入 y , $M_x'(y) = \perp$,因此 $P(M_x') = P(M_{\infty}) = 0$

因此: $\text{HALT}(M, x) = 1 \implies P(M_x') = 1$, $\text{HALT}(M, x) = 0 \implies P(M_x') = 0$. 因此,若 P 是可计算的,则停机问题是可计算的.而停机问题不可计算,故 P 也不可计算

Rice 定理表明:任何关于程序功能的非平凡问题都是不可判定的.

即,不存在一个通用的算法,能够针对任意程序,判定其是否具有某种涉及程序行为(而非代码结构)的性质,只要这个性质不是显然对所有程序成立或都不成立.如,我们无法编写一个程序,判定某个代码中的循环是否会终止,或是判断某一行代码是否冗余

Quine Problem

考虑这样一件事情:能否写出一个 python 程序,其输出自身的代码?

一个朴素的想法是:使用 `print` 函数不断输出上一条代码:

```
def f():
    print("def f():")
    print("print("def f():")")
    print("print("print("def f():")")")
    ... ..
```

这似乎陷入了一个循环,告诉我们输出自身代码这个事情是反直觉的.事实上,这是一个自引用(Self_Reference)或自复制(Quine)问题,在可计算性理论当中,我们关心的是:是否存在一个图灵机,它能将自己的编码输出在纸带上?

我们构造一个图灵机 M ,其将自身的编码 $\langle M \rangle$ 输出在纸带上.

将 M 分成 A, B 两部分:

- A :将 B 的编码 $\langle B \rangle$ 输出在纸带上
- B :将 A 的编码 $\langle A \rangle$ 输出在纸带上
- 最后交换 $\langle A \rangle$ 和 $\langle B \rangle$ 的位置

这样仍会导致问题:定义 A 前需要先定义 B ——毕竟先定义了 B 才能写出 $\langle B \rangle$,定义 B 时也会遇到一样的问题.

因此我们需要定义一个函数 $q: \{0, 1\}^* \rightarrow \{0, 1\}^*$,对于一个输入 w ,其输出一个图灵机的编码: $q(w) = \langle M_w \rangle$.图灵机 M_w 的功能为:对于任意的输入 x ,其将 w 输出到纸带上.易知:函数 q 是可计算的
接下来用 q 来构造 B :

$B =$ on input w :

- compute $q(w)$
- write $q(w)$ to the tape and swap it with w , we get $q(w)w$ on the tape

接下来用 B 定义 A :

$A =$ on any input:

- erase the input
- write $\langle B \rangle$ to the tape

在这样的定义下,依次运行 A 和 B ,纸带上的内容会这样变化:

$$\boxed{(\text{input}) \ x} \rightarrow \boxed{\langle B \rangle} \rightarrow \boxed{q(\langle B \rangle) \langle B \rangle}$$

而回顾函数 q 的定义会发现: $q(\langle B \rangle) = \langle A \rangle$,于是,我们构造了一个可以输出自身编码的图灵机.

如果图灵机能输出自身编码,我们也可以在面对输入 x 时,先获得自身的编码,然后计算一个有关 $\langle M \rangle$ 和 x 的函数 $f(\langle M \rangle, x)$

Recursion Theorem

Recursion Theorem

For any TM T that takes an input $\langle M \rangle$ and x , there exists a TM R such that for any $x \in \{0, 1\}^*$, $R(x) = T(\langle R \rangle, x)$

换句话说:我们可以构造一个程序 R ,其在输入 x 上的行为等价于图灵机 T 以 R 自身的代码和 x 作为输入时的行为.

以下内容来自 DeepSeek:

想象一个机器人装配线:

- 机器人 T : 拿到一个机器人的蓝图和某个任务 x , 执行某种操作
- 递归定理断言, 存在一个机器人 R , 但给他任务 x 时, 其可以执行以下操作:
 - 生成自己的完整蓝图 $\langle R \rangle$
 - 将自己的蓝图和任务 x 交给 T
 - 执行 T 的操作

而 R 不需要预先知道自己的蓝图, 他可以在运行时构造出来

该定理表明: 任何图灵机都可以在计算中获得自身的编码, 并利用这个编码进行后续计算. 其允许我们在构造图灵机时, 假设该图灵机已经有了自己的编码, 从而简化许多自引用结构的构造.

Quine Python

开头提到的“输出自身代码的 python 程序”, 其中一个实现为:

```
s = 's = %r\nprint(s%s)\nprint(s%s)'\nprint(s%s)
```

例如, 我们可以利用递归定理重新证明停机问题的不可计算性.

HALT is uncomputable

假设停机问题是可计算的, 构造一个图灵机 M :

M = on input w :

- obtain $\langle M \rangle$
- compute $\text{HALT}(M, w)$
- if $\text{HALT}(M, w) == 1$, loop forever
- else, halt

Then, for any w , M halts on w iff $\text{halt}(M, w) = 0$, 这就产生了矛盾

因此停机问题是不可计算的

利用这个定理还能证明一些其他看起来比较麻烦的问题是不可计算的.

We say a TM M is minimal if for any M' functionally equivalent to M , $|\langle M \rangle| \leq |\langle M' \rangle|$

MIN 问题: 输入一个图灵机的编码 $\langle M \rangle$, 判断 $\langle M \rangle$ 是否是极小的, 即不存在功能等价但编码更短的图灵机. 该问题是不可计算的

MIN problem is uncomputable

$$\text{MIN}(M) = \begin{cases} 1 & \text{if } M \text{ is minimal} \\ 0 & \text{otherwise} \end{cases}$$

假设存在图灵机 M_{MIN} 能判断 MIN 问题, 利用递归定理, 构造图灵机 R :

R = on input x :

- 获得自身的编码 $\langle R \rangle$
- 令 $n = |\langle R \rangle|$
- 遍历所有长度 $> n$ 的字符串 s (按长度和字典序):
 - 用 M_{MIN} 检查 s ,若 $M_{\text{MIN}} = 0$,则继续下一个
 - 若 $M_{\text{MIN}}(s) = 1$,则尝试将 s 解释为某个图灵机 M_s 的编码,模拟运行 $M_s(x)$
 - 一旦模拟得到输出, R 就输出与 $M_s(x)$ 相同的结果并停机.
- 由于我们假设对任何功能都存在极小机器,且图灵机 R 的行为被上述的最后一步定义,那么 R 最终会找到某个功能等价于自己的、编码更长的极小机器 M_s ,并模仿它的输出

这样就产生了矛盾: R 找到了一个功能与自己等价的但编码更长的极小机器 M_s ,而既然 M_s 编码更长,其就不可能是极小的,与 $M_{\text{MIN}}(s) = 1$ 矛盾.因此,判定机 M_{MIN} 不存在

Fixed Point Theorem

程序变换的不动点定理是递归定理的一个直接推论,其揭示了一个深刻的事实:对于任意可计算的程序变换(如编译器优化、代码混淆、翻译等),总存在一个程序,其经过变换后得到的程序与原程序语义等价.

Fixed pointed

令 $t: \{0, 1\}^* \rightarrow \{0, 1\}^*$ 为一个可计算的变换函数(例如,一个编译器优化过程),那么存在一个图灵机 F ,使得 $t(\langle F \rangle)$ 是某个图灵机 M 的编码,且 M 与 F 功能等价.

即:对于任意可计算的程序变换 t ,总存在一个程序 F ,其经过变换后的代码 $t(\langle F \rangle)$ 所描述的程序,与原来的 F 行为完全一样.

proof

给定一个可计算的变换 t ,我们可以用下面的方法构造 F :

- 定义一个辅助图灵机 T
 - T 接受输入:一个图灵机 M 的编码 $\langle M \rangle$ 和一个普通输入 x
 - T 的功能为:计算 $t(\langle M \rangle)$ 得到一个新的图灵机的编码 $t(\langle M \rangle)$,然后模拟这个新图灵机的运行,并输出结果
 - 形式化表述为: $T(\langle M \rangle, x) = U(t(\langle M \rangle), x)$
- 接下来我们应用递归定理:
 - 递归定理指出:存在图灵机 F 使得对所有 x ,都有 $F(x) = T(\langle F \rangle, x)$
 - 代入前面的 T 的定义可得: $F(x) = U(t(\langle F \rangle), x)$
- 而 $U(t(\langle F \rangle), x)$ 正是 $t(\langle F \rangle)$ 在输入 x 上的输出.因此 F 与 $t(\langle F \rangle)$ 在所有输入 x 上行为等价,故二者语义等价.

例如,假设有一个变换 t 为:在输入的程序开头插入代码 `print("Hello World!")`,定理保证存在一个 F ,其在插入代码前后行为不变,如:

- F 在运行前先检查自己的源代码(递归定理保证可以做到这一点)
- 如果开头有 `print("Hello World!")`,则只打印一次 Hello World!(保证插入前后打印的次数不变)

Godel's Incompleteness Theorem

除了递归定理之外,另一个重要的定理是哥德尔不完备定理.

希尔伯特计划希望为数学建立一个完备(所有真陈述都可证)、一致(没有矛盾)的公理系统.哥德尔不完备性定理粉碎了这个梦想

Gödel's incompleteness theorem

一个证明系统 V 对某个命题集合 T 而言:

- 有效性(Effectiveness):设 V 是一个图灵机,给定命题 x 和候选证明 t ,能在有限步内判定 t 是否是 x 的有效证明.
- 可靠性(Soundness):如果 $V(x, t) = 1$,那么 x 必须为真(即 $x \in T$),即证明不会证出假命题.
- 完备性(Completeness):如果 x 为真,那么存在某个 t 使得 $V(x, t) = 1$,即所有真命题都可证.

哥德尔定理指出:对于足够强(足以描述算术)且一致的公理系统,不存在同时满足可靠性和完备性的系统.

Proof

将其归约到 HALTONZERO 问题.令 T 为所有形如“图灵机 M 在输入 0 上停机”或“图灵机 M 在输入 0 上不停机”的命题的集合.如果 T 存在一个可靠且完备的证明系统 V ,那么 HALTONZERO 就是可判定的,这与其不可判定性矛盾

假设 V 是 T 的可靠且完备的证明系统.

要判定“ M 在 0 上是否停机”,构造算法:

并行地执行两个枚举过程:

- 枚举所有可能的字符串 t ,作为候选证明.用 V 检查 t 是否是“ M 在 0 上停机”的证明.如果找到,输出“停机”.
- 枚举所有可能的字符串 t ,作为候选证明.用 V 检查 t 是否是“ M 在 0 上不停机”的证明.如果找到,输出“不停机”.

由于 V 是可靠且完备的,命题“ M 在 0 上停机”要么真要么假,且必定存在一个证明 t 被枚举到.

因此,这个算法总能停机并给出正确答案.但这意味着我们解决了停机问题,矛盾.

所以,这样的 V 不存在.即,对于这个具体的真命题集合 T ,不存在可靠且完备的证明系统.

Quantified Integer Statement

量化整数语句(quantified integer statement)是在整数论域上,使用变量、常量、算数运算(+, ×)、关系(=, >, <)、逻辑连接词(∧, ∨, ¬)和量词(∀, ∃)构成的命题

example

- 费马大定理:

$$\neg \exists a \exists b \exists c \exists n, (a > 0) \wedge (b > 0) \wedge (c > 0) \wedge (n > 2) \wedge (a^n + b^n = c^n)$$

- 孪生素数猜想:

$$\forall n \exists p \forall x \forall y, (n > 2) \wedge (p > n) \wedge (xy > 1) \wedge (xy \neq p) \wedge (xy \neq pn) \wedge (x > 1) \wedge (y > 1)$$

我们可以证明真值判定问题是不可判定的,即:判定一个给定量化整数语句 Φ 是否为真(即 EVAL 函数)的问题是不可判定的.

$$\text{EVAL}(\Phi) = \begin{cases} 1 & \text{if } \Phi \text{ is true} \\ 0 & \text{otherwise} \end{cases}$$

EVAL is uncomputable

证明思路同样是归约到停机问题.对于任意的图灵机 M 和输入 x ,构造一个量化整数语句 $\Phi_{M,x}$,使得:

$$M \text{ halt on } x \iff \Phi_{M,x} = 1$$

令 T_{QIS} 为所有为真的量化整数语句的集合,由 EVAL 的不可判定性可以直接推出:不存在一个可靠且完备的证明系统来验证 T_{QIS} ,否则我们就可以通过系统性地搜索证明来判定 EVAL.

其可以看作是哥德尔不完备性定理的一个具体而强烈的实例

Running Time

Running Time of Turing Machine

Running Time

设 $T: \mathbb{N} \rightarrow \mathbb{N}$ 是一个函数,称图灵机 M 的运行时间为 $T(n)$,如果对于足够大的 n 和任意的 $x \in \{0, 1\}^n$, M 在输入 x 上都最多在 $T(n)$ 步内停机

基于此,我们可以定义一个布尔函数的集合 $\text{TIME}_{\text{TM}}(T(n))$:

$$\text{TIME}_{\text{TM}}(T(n)) = \{\text{boolean function } F : F \text{ is computable by some TM with running time } T(n)\}$$

运行时间函数通常是一个 **nice function**,即满足:

- $T(n) \geq n$ (至少要读取输入)
- $T(n)$ 是非递减的,即对于越长的 n ,运行时间 $T(n)$ 也应当越长
- $T(n)$ 本身可以在 $O(T(n))$ 时间内计算

P and EXP

多项式时间:

$$P = \cup_{c=1,2,3,\dots} \text{TIME}_{\text{TM}}(n^c)$$

指数时间:

$$\text{EXP} = \cup_{c=1,2,3,\dots} \text{TIME}_{\text{TM}}(2^{n^c})$$

我们有: $P \subsetneq \text{EXP}$

Running Time of NAND-RAM Program

Running Time

设 $T: \mathbb{N} \rightarrow \mathbb{N}$,称 NAND-RAM 程序 P 的运行时间为 $T(n)$,如果对于所有足够大的 n 和任意的 $x \in \{0, 1\}^n$, P 在输入 x 上都在执行最多 $T(n)$ 行后停机

基于此,我们可以定义一个布尔函数的集合 $\text{TIME}_{\text{RAM}}(T(n))$:

$$\text{TIME}_{\text{RAM}}(T(n)) = \{F : F \text{ is computable by some NAND-RAM program with running time } T(n)\}$$

TM and NAND-RAM

equality of TM and NAND-RAM

对于一个 nice function $T(n)$,有:

$$\text{TIME}_{\text{TM}}(T(n)) \subset \text{TIME}_{\text{RAM}}(10T(n)) \subset \text{TIME}_{\text{TM}}(T(n)^4)$$

proof sketch

- TM $\rightarrow$ NAND-RAM: TM 的一步可以用常数条 RAM 指令模拟
- NAND-RAM $\rightarrow$ TM: NAND-RAM 的一行可以用 TM 的多项式步模拟

推论:在 RAM 模型下定义的 P 和 EXP 与 TM 模型下的 P 和 EXP 是多项式等价的

Universal TM and Universal NAND-RAM Program

通过先前的学习我们知道:存在通用图灵机 U ,使得对于任意图灵机 M 和输入 x ,都有:

$$U(M, x) = M(x)$$

类似的,也存在通用 NAND-RAM 程序 U ,使得对于任意 NAND-RAM 程序 P 和输入 x ,都有:

$$U(P, x) = P(x)$$

若 P 在 x 上的运行时间为 $T(|x|)$,则 U 模拟 P 的运行时间为:

$$O(|\langle P \rangle|^b \cdot T(|x|))$$

其中 b 是常数,即模拟的开销是多项式倍的

Time Hierarchy Theorem

 **Time Hierarchy Theorem** 对于任意 nice function $T : \mathbb{N} \rightarrow \mathbb{N}$,存在函数 $F : \{0, 1\}^n \rightarrow \{0, 1\}$ 满足:

$$F \in \text{TIME}_{\text{RAM}}(T(n) \log n) \setminus \text{TIME}_{\text{RAM}}(T(n))$$

即存在函数 F 无法被一个 NAND-RAM 程序在 $T(n)$ 内解决,但是存在一个 NAND-RAM 程序可以在 $T(n) \log n$ 内解决

proof sketch

定义有界停机问题:

$$\text{HALT}_T(\langle P \rangle, x) = \begin{cases} 1 & \text{if } |P| \leq \log \log |x| \text{ and } P \text{ halts on } x \text{ in } 100T(|P| + |x|) \text{ steps} \\ 0 & \text{otherwise} \end{cases}$$

- 断言 1: $\text{HALT}_T \in \text{TIME}_{\text{RAM}}(T(n) \log n)$

proof of assertion 1

若 $|P| > \log \log |x|$, 直接返回 0, 否则计算 $T_0 = T(|P| + |x|)$, 模拟 P 在 x 上运行最多 $100T_0$ 步。模拟开销为 $O(|P|^b T_0)$

- 由于 $|P| \leq \log \log |x|$, 且输入总长度 $n = |P| + |x|$, 有 $|P|^b = O((\log \log n)^b) = o(\log n)$
- 总模拟开销为: $O(T_0 \cdot (\log \log n)^b) = o(T(n) \log n)$

- 断言 2: $\text{HALT}_T \notin \text{TIME}_{\text{RAM}}(T(n))$

proof of assertion 2

假设存在运行时间为 $O(T(n))$ 的程序 A 计算 HALT_T 。构造程序 B 如下:

1. 输入: 字符串 z
2. 计算 $m = |z|$
3. 找到最小的 n 使得 $|z| \leq \log \log n$ (这样的 n 存在且满足 $n = \Omega(2^{2^{|z|}})$)
4. 令 x 为 z 填充到长度为 n (后面补 0)
5. 运行 A 于输入 (z, x) , 得到输出 b
6. 输出 $1 - b$

分析 B 的运行时间: 步骤 3 可在 $O(n)$ 内完成, n 约为 $2^{2^{|z|}}$ 。步骤 5 中 A 的运行时间为 $O(T(|z| + n))$ 。由于 T 是 nice function, $T(|z| + n) = T(O(n)) = O(T(n))$ 。但 B 在输入 z 上的行为与 A 对 (z, x) 的预测矛盾:

- 如果 $A(z, x) = 1$ (预测 z 在 x 上 $100T(|z| + |x|)$ 步内停机), 则 $B(z) = 0$, 但实际上 B 会运行超过 $100T(|z| + |x|)$ 步 (因为要模拟 A , 而 A 运行时间至少为 $T(|z| + |x|)$), 矛盾。
- 如果 $A(z, x) = 0$, 则 $B(z) = 1$, 但 B 在 $100T(|z| + |x|)$ 步内停机, 矛盾。

由时间层次定理可知:

- $P \subsetneq \text{EXP}$, 即存在函数在指数时间可计算但多项式时间不可计算
- 时间资源是严格分层的: 给更多时间就能计算更多函数

Complexity of Circuit and Time

对于无限函数 $F: \{0, 1\}^* \rightarrow \{0, 1\}$, 我们可以分析其对不同输入长度的电路复杂度:

定义: $F_n: \{0, 1\}^n \rightarrow \{0, 1\}$ 是 F 在 n 位输入上的限制, 即将 F 的输入限制为 n 位。

- 称 $F_n \in \text{Size}_n(T(n))$ 如果存在大小为 $T(n)$ 的电路 (NAND-CIRC 程序) 计算 F_n 。
- 称 $F \in \text{Size}(T(n))$ 如果对于每个 n , 都有 $F_n \in \text{Size}_n(T(n))$ 。

等价地:

- $F \in \text{Size}(T(n))$: 对于每个 n , 存在 NAND-CIRC 程序 P_n , 对于所有 $x \in \{0, 1\}^n$, $P_n(x) = F(x)$, 且 P_n 的大小 (门数) 不超过 $T(n)$ 。
- $F \in \text{TIME}(T(n))$: 存在图灵机 M , 对于所有 n 和 $x \in \{0, 1\}^n$, $M(x) = F(x)$ 且在 $T(n)$ 步内停机。

时间与电路复杂度的关系

对于任意 nice function $T : \mathbb{N} \rightarrow \mathbb{N}$, 有

$$\text{TIME}(T(n)) \subset \text{Size}(T(n)^3)$$

 **proof** 证明思路:

给定图灵机 M 、步数 t 和输入长度 n , 存在电路 C (NAND-CIRC 程序) 具有 n 个输入、 $O(t^2)$ 个门和 1 个输出, 使得对于任意 $x \in \{0, 1\}^n$:

$$C(x) = \begin{cases} 1 & \text{if } M(x) \text{ halts in } t \text{ steps} \\ 0 & \text{otherwise} \end{cases}$$

构造方法: 将图灵机的格局(configuration)序列展开, 每个格局用 $O(t)$ 个门表示, 总共 t 个格局. 每个格局到下个格局的转移是局部函数, 可以用 $O(1)$ 个门实现. 因此总门数为 $O(t^2)$.

要计算 M 在 $T(n)$ 步内的输出, 需要 $O(T(n)^2)$ 个门. 进一步优化可得到 $O(T(n)^3)$.

多项式大小电路类:

$$P_{\text{poly}} = \bigcup_{c=1,2,3,\dots} \text{Size}(n^c)$$

关系:

1. $P \subsetneq P_{\text{poly}}$
2. 存在不可计算函数在 P_{poly} 中

 **proof** 存在不可计算函数在 P_{poly} 中

考虑不可计算函数 $F : \{0, 1\}^* \rightarrow \{0, 1\}$, 使得 F_n 要么是恒等函数, 要么是全零函数, 根据 n 是否属于某个不可判定的集合(如停机问题的编码)决定. 对于每个 n , F_n 要么是恒等函数(需要 $O(n)$ 个门), 要么是全零函数(需要 0 个门), 因此 $F \in \text{Size}(n) \subset P_{\text{poly}}$, 但 F 不可计算.

3SAT Problem

The 3SAT problem can be phrased as the function $3SAT : \{0, 1\}^* \rightarrow \{0, 1\}$ that takes as input a 3CNF formula ϕ and maps ϕ to 1 if there exists some assignment to the variables of ϕ that causes it to evaluate to true, and to 0 otherwise.

- 3CNF formula:

i.e. a formula of the form $C_0 \wedge C_1 \wedge \dots \wedge C_{m-1}$ where each C_i is the OR of three variables or their negations.

3SAT problem

$$3SAT((x_0 \vee \bar{x}_1 \vee x_2) \wedge (x_1 \vee x_2 \vee \bar{x}_3) \wedge (\bar{x}_0 \vee \bar{x}_2 \vee x_3)) = 1$$

since the assignment $x = 1101$ satisfies the input formula.

💡 Zero-One Linear Equations problem

The Zero-One Linear Equations problem corresponds to the function $01EQ : \{0, 1\}^* \rightarrow \{0, 1\}$ whose input is a collection E of linear equations in variables $x_0, x_1, \dots, x_{n-1}$, and the output is 1 iff there is an assignment $x \in \{0, 1\}^n$ of 0/1 values to the variables that satisfies all the equations. For example, if the input E is a string encoding the set of equations

$$x_0 + x_1 + x_2 = 2$$

$$x_0 + x_2 = 1$$

$$x_1 + x_2 = 2$$

then $01EQ(E) = 1$ since the assignment $x = 011$ satisfies all three equations

值得注意的是,这里的加法是整数加法,而不是布尔加法.由于变量取值只能为 0 或 1,方程 $x_0 + x_1 + x_2 = 2$ 意味着三个变量中恰好有两个为 1.

💡 Subset Sum Problem

Given a list of integers $x_0, \dots, x_{n-1} \in \mathbb{Z}$ and an integer $T \in \mathbb{Z}$. We need to determine whether or not there exists some set of the integers that sums up to T . That is, for $x_0, \dots, x_{n-1}, T \in \mathbb{Z}$, $SSUM(x_0, \dots, x_{n-1}, T) = 1$ iff there exists $S \subseteq [n]$ such that $\sum_{i \in S} x_i = T$.

Note that the input length for the subset sum problem is the length of string needed to encode all the numbers, which will be approximately $\lceil \log T \rceil + \sum_{i=0}^{n-1} \lceil \log x_i \rceil$, since encoding an integer x using the binary representation requires $\lceil \log x \rceil$ bits.

polynomial-time Reductions

假设 $F, G : \{0, 1\}^* \rightarrow \{0, 1\}$ 是两个布尔函数,一个从 F 到 G 的多项式时间规约表明 F 是不难于 G 的.

💡 Polynomial-time Reductions

Let $F, G : \{0, 1\}^* \rightarrow \{0, 1\}$. A reduction from F to G , which is denoted by $F \leq_p G$, is a polynomial-time computable function $R : \{0, 1\}^* \rightarrow \{0, 1\}^*$ such that for every $x \in \{0, 1\}^*$:

$$F(x) = G(R(x))$$

规约的基本性质:如果 $F \leq_p G$, 且 $G \in P$, 则 $F \in P$.

🔍 Proof of basic property

由于 $G \in P$, 存在一个多项式时间的算法 M_G 可以在 $O(n^c)$ 时间内判定 G 。由于 $F \leq_p G$, 存在一个多项式时间可计算函数 R , 其计算时间为 $O(n^k)$ 。

我们要为 F 构造一个多项式时间算法 M_F 。对于输入 x (长度为 n):

1. 运行 $R(x)$ 得到 y 。由于 R 是多项式时间的, $|y| \leq O(n^k)$, 且这一步花费 $O(n^k)$ 时间。
2. 运行 $M_G(y)$ 得到结果。这一步花费 $O(|y|^c) = O((n^k)^c) = O(n^{kc})$ 时间。

3. 输出 $M_G(y)$ 的结果。

总时间复杂度为 $O(n^k + n^{kc})$ ，这仍然关于 n 的多项式。因此 $F \in P$ 。

Transitivity of reductions. Since we think of $F \leq_p G$ as saying that F is “easier or equal in difficulty to” G , we would expect that if $F \leq_p G$ and $G \leq_p H$, then it would hold that $F \leq_p H$. Indeed this is the case:

Transitivity of polynomial-time reductions

For every $F, G, H : \{0, 1\}^* \rightarrow \{0, 1\}$, if $F \leq_p G$ and $G \leq_p H$, then $F \leq_p H$

proof

设 R_1 是从 F 到 G 的规约， R_2 是从 G 到 H 的规约。定义函数 $R(x) = R_2(R_1(x))$ 。

1. 正确性: $H(R(x)) = H(R_2(R_1(x))) = G(R_1(x)) = F(x)$ 。因此 x 在 F 中当且仅当 $R(x)$ 在 H 中。
2. 效率: R_1 耗时 $O(n^a)$ ， R_2 耗时 $O(n^b)$ 。计算 $R_1(x)$ 输出的长度最多为 $O(n^a)$ 。接着计算 R_2 的输入长度为 $O(n^a)$ ，因此耗时 $O((n^a)^b) = O(n^{ab})$ 。两个多项式的合成仍然是多项式，因此 R 是多项式时间可计算的。

Hardness of 01EQ and Subset sum

- $3SAT \leq_p 01EQ$



我们需要将一个 3CNF 公式 φ 转化为一组线性方程 E ，使得 φ 可满足当且仅当 E 有 0/1 解。

设 φ 有 n 个变量 $x_1, \dots, x_n$ 和 m 个子句 $C_1, \dots, C_m$ 。对于 E 中的变量，我们直接使用 $x_1, \dots, x_n$ 。

对于每个子句 $C_j = (l_1 \vee l_2 \vee l_3)$ ，其中 l_k 是文字（变量 x_i 或其非 $\bar{x}_i$ ）：我们希望表达 $l_1 + l_2 + l_3 \geq 1$ （至少一个为真）。在 01EQ 中，我们引入两个松弛变量 (slack variables) $y_{j,1}, y_{j,2}$ ，它们取值也限制为 $\{0, 1\}$ 。我们将该子句转化为如下线性方程（注意 $\bar{x}$ 转化为 $1 - x$ ）：

$$\text{val}(l_1) + \text{val}(l_2) + \text{val}(l_3) - y_{j,1} - y_{j,2} = 1$$

其中 $\text{val}(x_i) = x_i$, $\text{val}(\bar{x}_i) = 1 - x_i$ 。

分析：

- 若 l_1, l_2, l_3 中有 1, 2, 或 3 个为真，则它们的和为 1, 2, 或 3。我们可以通过选择适当的 $y_{j,1}, y_{j,2}$ (和为 0, 1, 或 2) 来满足方程。

- 和为 1: 取 $y_{j,1} = 0, y_{j,2} = 0$ 。
- 和为 2: 取 $y_{j,1} = 1, y_{j,2} = 0$ 。
- 和为 3: 取 $y_{j,1} = 1, y_{j,2} = 1$ 。
- 若全为假 (和为 0), 方程变为 $0 - y_{j,1} - y_{j,2} = 1$, 这在 $y \in \{0,1\}$ 时无解。

因此, 方程组 E 有解当且仅当每个子句都被满足。该转换是多项式时间的 (方程数量与子句数量成线性关系)。

$$\bullet \quad 3SAT \leq_p SSUM$$



给定 3CNF 公式 φ , 我们需要构造集合 $S = \{y_1, \dots, y_k\}$ 和目标数 T 。我们要利用十进制表示 (或足够大的基数如 4, 以避免进位) 来编码约束。

假设 φ 有 n 个变量和 m 个子句。我们构造的数字将有 $n + m$ 位。前 n 位对应变量约束, 后 m 位对应子句约束。

1. 构造数字:

- 对于每个变量 x_i :
 - 创建两个数 v_i (代表选 $x_i = 1$) 和 v'_i (代表选 $x_i = 0$)。
 - v_i 在第 i 位为 1, 在 x_i 出现的每个子句 C_j 对应的位上也为 1。
 - v'_i 在第 i 位为 1, 在 $\bar{x}_i$ 出现的每个子句 C_j 对应的位上也为 1。
- 对于每个子句 C_j :
 - 创建两个松弛数 $s_{\{j,1\}}, s_{\{j,2\}}$ 。
 - 它们在变量位全为 0, 仅在子句 C_j 对应的位上分别为 1 和 1 (或是其他组合, 目的是凑数)。

2. 构造目标 T :

- 前 n 位 (变量位) 每一位都是 1。这强制对于每个 i , 必须恰好在 v_i 和 v'_i 中选一个 (保证变量赋值的一致性)。
- 后 m 位 (子句位) 每一位都是 3 (假设基数大于 3)。
 - 如果一个子句被满足, 其对应位的和可能是 1, 2, 或 3 (取决于有几个文字为真)。
 - 利用松弛数 $s_{\{j,1\}}, s_{\{j,2\}}$, 我们可以把 1 补成 3, 把 2 补成 3, 3 则不需要补。
 - 如果子句未满足 (和为 0), 即使选了所有松弛数也无法达到 3。

这样, 存在子集和为 T 当且仅当存在满足 φ 的赋值。整个构造过程是多项式时间的。

NP

多项式时间可验证:



polynomial-time verifiable $F : \{0,1\}^* \rightarrow \{0,1\}$ is polynomial-time verifiable if exists a TM V with polynomial running time such that for any $x \in \{0,1\}^*, F(x) = 1$ iff exists $t \in \{0,1\}^*$

$$\begin{cases} V(x, t) = 1 \\ |t| \leq \text{poly}(|x|) \end{cases}$$

其中 t 称为证书或见证.例如:对于3SAT问题,证书 t 是一个满足赋值,验证机 V 检查该赋值是否满足公式. 3SAT 对应的 V :

Algorithm 3: Verifier for 3SAT

```

1:  ▷ Input: 3CNF formula  $\varphi$  on  $k$  variables and with  $m$  clauses, string  $w \in \{0, 1\}^k$ 
2:  ▷ Output: 1 iff  $w$  is a valid assignment to the variables of  $\varphi$ 
3:  for  $j \in [m]$  do
4:    ▷ Let  $l_1 \vee l_2 \vee l_3$  be the  $j$ -th clause of  $\varphi$ 
5:    if  $w$  violates all three literals then
6:      return 0
7:    end
8:  end
9:  return 1

```

💡 NP

We say that $F : \{0, 1\}^* \rightarrow \{0, 1\}$ is in **NP** if there exists some integer $a > 0$ and $V : \{0, 1\}^* \rightarrow \{0, 1\}$ such that $V \in \mathbf{P}$ and for every $x \in \{0, 1\}^*$,

$$F(x) = 1 \iff \exists w \in \{0, 1\}^{n^a} \text{ s.t. } V(xw) = 1$$

- x 是问题的输入实例
- w 是一个证书(witness),其长度是 $|x|$ 的多项式,即 $|w| = n^a$
- V 是一个多项式时间可计算的验证函数,其接受 x 和 w 作为输入,输出1表示接受,即证书有效,输出 0 表示拒绝

换句话说:NP 是一个布尔函数的集合,对于其中的函数,给出一个解,存在一个验证机可以在多项式时间内验证该解是否正确.

📐 Basic facts about NP

$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{EXP}$

🔍 **P ⊆ NP** 设 $F \in \mathbf{P}$. 定义函数 V :

$$V(x0^n) = \begin{cases} 1 & \text{if } n = |x| \text{ and } F(x) = 1 \\ 0 & \text{otherwise} \end{cases}$$

由于 $F \in \mathbf{P}$, 故易得 $V \in \mathbf{P}$; 令 $x \in \{0, 1\}^n$, 若 $F(x) = 1$, 则 $V(x0^n) = 1$. 而若 $F(x) = 0$, 则 $\forall w \in \{0, 1\}^n, V(xw) = 0$. 因此, 令 $a = 1$ (i.e. $w \in \{0, 1\}^{n^1}$), 则 V 符合 NP 定义中的条件:

$$F(x) = 1 \iff \exists w \in \{0, 1\}^{n^a} \text{ s.t. } V(xw) = 1$$

故 $F \in \mathbf{NP}$.

NP $\subseteq$ EXP

设 $F \in \text{NP}$, 令 V 为一个满足 NP 定义中的等式的多项式时间可计算函数, a 为相关的常数. 于是, 对于给定的字符串 $x \in \{0, 1\}^n$, 通过枚举所有的 2^{n^a} 个字符串 $w \in \{0, 1\}^{n^a}$ 并检查 $V(xw)$ 是否为 1, 我们可以在时间 $\text{poly}(n) \cdot 2^{n^a} = o(2^{n^{a+1}})$ 内检查 $F(x)$ 是否为 1.

若 $F(x) = V(xw) = 1$, 则返回 1, 若 $\forall w, V(xw) = 0$ 则返回 0. 该算法会在最多 $\exp(\text{length})$ 内给出结果并对任意的 x 返回 $F(x)$

简而言之, 该算法暴力搜索所有可能的证书: 由于证书长度是多项式的, 那么可能的证书数量是指数的, 对每个证据用多项式时间验证.

NP does not mean non-polynomial. 若一个函数 F 在 NP 中, 并不能说明其是难以计算的, 这只意味着其是容易被验证的.

theorem: 3SAT in P iff P = NP

NP-hardness and NP-completeness

Let $G : \{0, 1\}^* \rightarrow \{0, 1\}$. We say that G is **NP hard** if for every $F \in \text{NP}, F \leq_p G$.

We say that G is **NP complete** if G is **NP hard** and $G \in \text{NP}$.

Lemma: Let G be a NP-complete function. if $G \in \text{NP}$, then P = NP

Cook-Levin theorem:

Cook-Levin Theorem

For every $F \in \text{NP}, F \leq_p 3\text{SAT}$

然而, 3SAT 问题并不是一个十分特殊的问题, 事实上, 将这个定理中的 3SAT 问题换成 *LONGPATH*, *QUADEQ* 或是 *MAXCUT* 都是成立的.

proof

证明通过一个规约链完成:

$$F \leq_p \text{NANDSAT} \leq_p 3\text{NAND} \leq_p 3\text{SAT}$$

NANDSAT problem

NANDSAT 问题按照如下规则定义:

- 输入为一表示一个 NAND-CIRC 程序的字符串 Q
- 输出为 1 iff $\exists w \in \{0, 1\}^n, Q(w) = 1$

有以下结论:

- $NANDSAT \in NP$: NAND-CIRC 程序本就可以在多项式时间内运行完成
- $NANDSAT$ is NP hard: 若是证明了这点,也就说明了 $F \leq_p NANDSAT$ (由 NP hard 的定义)

 $NANDSAT$ is NP-hard

3NAND problem

3NAND问题按照如下规则定义:

- 输入为一个变量集合 $\{z_0, \dots, z_{r-1}\}$ 上的逻辑公式 Ψ , Ψ 为若干个形如 $z_i = NAND(z_j, z_k)$ 的约束的 AND. 如:

$$\Psi = (z_3 = NAND(z_0, z_2)) \wedge (z_4 = NAND(z_3, z_1))$$

- 输出为 1 iff $\exists z \in \{0, 1\}^r$, 其满足所有约束

可以证明的是: $NANDSAT \leq_p 3NAND$

 $NANDSAT \leq_p 3NAND$

 $3NAND \leq_p 3SAT$

Random Algorithm and BPP

多项式等式验证

example

多项式等式验证问题, 给定两个 d 次多项式:

$$F(x) = \prod_{i=1}^d (x - a_i)$$

$$G(x) = \sum_{i=1}^d b_i x^i$$

判断 $F(x) = G(x)$ 是否对于所有的 x 成立.

- 确定性方法
 - 比较系数: 展开 $F(x)$ 得到各项系数, 将其与 $G(x)$ 的系数逐一比较, 这需要 $O(d^2)$ 时间

- 根验证:由于 $F(x)$ 有 d 个根 $a_1, \dots, a_d$,如果 $F(x) = G(x)$,那么 $G(a_i) = 0$ 对所有 i 成立,验证所有根需要 $O(d^2)$ 时间
- 随机化方法:

Algorithm 4: Compare F and G

```

1: ▷ input:  $F, G$ 
2: ▷ randomly choose a sample  $r$  from  $\{1, \dots, 100d\}$ 
3: ▷ Compute  $F(r)$  and  $G(r)$ 
4: if  $F(r) == G(r)$  then
5:   | return 1
6: end
7: return 0

```

分析:

- 运行时间: $O(d)$,因为只需要计算一个点上的值
- 正确性:
 - 如果 $F(x) = G(x)$,算法总是输出正确
 - 如果 $F(x) \neq G(x)$,设 $H(x) = F(x) - G(x)$,则 $H(x)$ 是非零多项式,次数最多为 d ,根据代数基本定理, $H(x)$ 最多有 d 个根,算法出错的概率即为 r 恰好是 $H(x)$ 的根的概率:

$$\Pr[H(r) = 0] = \frac{d}{100d} = \frac{1}{100}$$

改进:通过重复采样 k 次,可以将错误概率降至 $(\frac{1}{100})^k$

矩阵乘法验证

 example

矩阵乘法验证问题,给定三个 $n \times n$ 矩阵 A, B, C ,判断 $A \cdot B = C$ 是否成立

- 确定性方法:
 - 直接计算 $A \cdot B$,需要 $O(n^\omega)$ 时间,其中 $\omega \approx 2.373$,将其与 C 中的元素逐个比较
- 随机化方法:

Algorithm 5: Matrix multiplication

```

1: ▷ input:  $n \times n$  matrix  $A, B, C$ 
2: ▷ randomly choose a sample  $r$  which is a  $n$  dimensional 0/1 vector
3: ▷ Compute  $p = A \cdot (B \cdot \vec{r})$  and  $q = C \cdot \vec{r}$ 
4: if  $p == q$  then
5:   | return 1
6: end
7: return 0

```

分析:

- 运行时间: $O(n^2)$,包括三次矩阵-向量乘法
- 正确性:
 - 如果 $A \cdot B = C$,算法总是输出正确
 - 如果 $A \cdot B \neq C$,设 $D = A \cdot B - C$,则 D 是非零矩阵算法出错的条件是 $D \cdot \vec{r} = 0$,对于非零矩阵,

$$\Pr[D \cdot \vec{r} = 0] \leq \frac{1}{2}$$

- 改进:重复算法 k 次,每次独立随机选择 r ,如果每次结果都是 $p = q$,则输出“相等”,否则输出“不相等”.此时错误概率降低至 $(\frac{1}{2})^k$,取 $k = \log_2 n$,错误概率为 $\frac{1}{n}$,时间开销为 $O(n^2 \log(n))$

 $\Pr[D \cdot r = 0] \leq \frac{1}{2}$

设 D 是非零矩阵。考虑 D 的第一非零行（假设为第 i 行），记为 $d_1, d_2, \dots, d_n$ ，且 $d_j \neq 0$ 对某个 j 。

固定其他 r_k ($k \neq j$) 的值，考虑随机变量 r_j ：

$$D \cdot \vec{r} = \sum_{k=1}^n d_k r_k$$

可以写成：

$$d_j r_j + S = 0 \quad \text{其中} \quad S = \sum_{k \neq j} d_k r_k$$

对于固定的 S ，方程 $d_j r_j = -S$ 有唯一解 $r_j = -\frac{S}{d_j}$ （因为 $d_j \neq 0$ ）。

由于 r_j 在 $\{0, 1\}$ 上均匀分布，这个特定值出现的概率最多为 $\frac{1}{2}$ 。

随机计算模型

Randomized Turing Machine

随机图灵机是标准图灵机的扩展,具有两种转移方式:

$$\delta(q, \sigma) = \begin{cases} (q_1, \sigma_1, d_1) \\ (q_2, \sigma_2, d_2) \end{cases}$$

每一步,机器以相等的概率选择两个转移之一。

在程序上,我们可以扩展 NAND-TM Program,添加随机指令:

? = RAND()

这条指令将 0 或 1 以相等的概率赋值给变量?.这样就得到了 RNAND-TM Program.

BPP

BPP(Bounded-error Probabilistic Polynomial time)

函数 $F: \{0, 1\}^* \rightarrow \{0, 1\} \in \text{BPP}$,如果存在多项式 $p(n)$ 和随机图灵机 M ,使得对于所有输入 $x \in \{0, 1\}^*$:

- M 在 $p(|x|)$ 步内停机
- $\Pr[M(x) = F(x)] \geq \frac{2}{3}$

其中概率是 M 的内部随机选择

另:常数 $\frac{2}{3}$ 可以替换为 $\frac{1}{2} + \epsilon$,其中 $\epsilon > 0$ 是任意常数,通过重复运行和多数表决,可以将成功概率提高到任意接近 1.

Amplification Lemma

放大引理: 设 $F: \{0, 1\}^* \rightarrow \{0, 1\}$, 如果存在多项式时间随机图灵机 P 和常数 $\delta < \frac{1}{2}$, 使得对于所有 $x \in \{0, 1\}^*$:

$$\Pr[P(x) = F(x)] \geq 1 - \delta$$

那么对于任意多项式 $q(n)$, 存在多项式时间随机图灵机 P' , 使得:

$$\Pr[P'(x) = F(x)] \geq 1 - 2^{-q(|x|)}$$

proof

构造 P' : 在输入 x 上独立运行 P 共 k 次, 取多数输出作为答案。

设 X_i 为指示变量, $X_i = 1$ 表示第 i 次运行输出正确结果。则 $E[X_i] \geq 1 - \delta$ 。

令 $S = \sum_{i=1}^k X_i$ 。我们需要 $S > \frac{k}{2}$ 以保证多数表决正确。

由切尔诺夫不等式, 当 $\delta < \frac{1}{2}$ 时, 错误概率以指数速度下降:

$$\Pr\left[S \leq \frac{k}{2}\right] \leq e^{-ck}$$

其中 $c > 0$ 是常数。

取 $k = O(q(n))$, 即可使错误概率小于 $2^{-q(n)}$ 。

BPP 的等价定义: $F \in \text{BPP}$ iff $\exists G \in \text{P}$ 和多项式 $p(n)$, 使得对于所有 $x \in \{0, 1\}^*$:

$$\Pr_{r \in \{0, 1\}^{p(|x|)}}[G(x, r) = F(x)] \geq \frac{2}{3}$$

其中 r 是均匀随机串。

proof

$\Rightarrow$: 若 $F \in \text{BPP}$, 则存在随机图灵机 M 在时间 $q(n)$ 内以概率 $\frac{2}{3}$ 计算 F , 将 M 的随机选择编码为字符串 $r \in \{0, 1\}^{q(n)}$, 定义 $G(x, r)$ 为确定性模拟 $M(x)$ 使用随机串 r 的输出 $\Leftarrow$: 如果存在这样的 G , 随机算法只需随机选择 r , 然后计算 $G(x, r)$

BPP 与其他复杂类的关系

$$P \subseteq \text{BPP} \subseteq \text{EXP}$$

proof

- $P \subseteq \text{BPP}$: 确定性算法是随机算法的特例(忽略随机性)
- $\text{BPP} \subseteq \text{EXP}$: 如果 $F \in \text{BPP}$, 则存在多项式时间算法, 我们可以枚举所有可能的随机串(指数多个), 统计多数输出, 这需要指数时间, 但仍是有限时间

$$\text{BPP} \subseteq P_{\text{poly}}$$

proof

设 $F \in BPP$, 由放大引理, 存在随机算法 A 对于输入 $x \in \{0, 1\}^n$, 使用 $m = \text{poly}(n)$ 个随机比特, 且错误概率小于 2^{-n}

固定输入长度 n , 考虑所有可能的随机串 $r \in \{0, 1\}^m$, 对于每个 $x \in \{0, 1\}^n$, 最多有 $2^{-n} \cdot 2^m$ 个随机串使得 $A(x, r)$ 出错

因此, 对于所有 $x \in \{0, 1\}^n$ 都出错的随机串 r 的数量最多为:

$$2^n \cdot 2^{-n} \cdot 2^m = 2^m$$

这等于所有随机串的数量, 说明存在一个随机串 r^* 使得对于所有 $x \in \{0, 1\}^n$, $A(x, r^*)$ 都正确
将这个固定的 r^* 硬编码到电路中, 我们得到一个多项式大小的电路计算 F_n , 因此 $F \in P_{\text{poly}}$

伪随机生成器

Pseudorandom Generator

设 $G: \{0, 1\}^\ell \rightarrow \{0, 1\}^m$ 是一个函数, 其中 $\ell < m$, 称 G 是一个 (T, ε) -伪随机生成器, 如果对于任意具有 m 个输入, 1 个输出且大小最多为 T 的电路 C :

$$\left| \Pr_{s \in \{0, 1\}^\ell} [C(G(s)) = 1] - \Pr_{r \in \{0, 1\}^m} [C(r) = 1] \right| \leq \varepsilon$$

即: G 将短的随机种子 s 长 ℓ 扩展为长的为随机串 $G(s)$ (长 m), 使得任何大小不超过 T 的电路无法区分 $G(s)$ 和真正的随机串 r

Optimal PseudoRandom Generator

如果存在伪随机生成器 $G: \{0, 1\}^\ell \rightarrow \{0, 1\}^{2^\ell}$ 且 $\varepsilon = 2^{-\ell}$, 则称其为“最优伪随机生成器”。

最优伪随机生成器可以将 ℓ 位的随机性扩展到 2^ℓ 位的伪随机性, 这是可能达到的最大扩展 (因为种子只有 ℓ 位, 最多产生 2^ℓ 个不同的输出)

定理

伪随机生成器与去随机化

如果存在多项式时间可计算的伪随机生成器 $G: \{0, 1\}^{O(\log n)} \rightarrow \{0, 1\}^{\text{poly}(n)}$ 且 $\varepsilon = \frac{1}{n}$, 那么 $P = BPP$ 。

 proof 设 $F \in BPP$, 存在多项式时间算法 A 使用 $m = \text{poly}(n)$ 个随机比特。
使用伪随机生成器 G , 我们可以将随机种子长度从 m 减少到 $O(\log n)$ 。

新算法: 在输入 x 上, 枚举所有可能的种子 $s \in \{0, 1\}^{O(\log n)}$ (只有 $\text{poly}(n)$ 个), 计算 $A(x; G(s))$, 取多数输出。

由于 G 是伪随机的，这个确定性算法的输出与 A 非常接近。具体地，如果 A 的正确概率为 $\frac{2}{3}$ ，则新算法的正确概率至少为 $\frac{2}{3} - \varepsilon > \frac{1}{2}$ （通过放大引理可进一步提高）。

因此 $F \in P$ 。